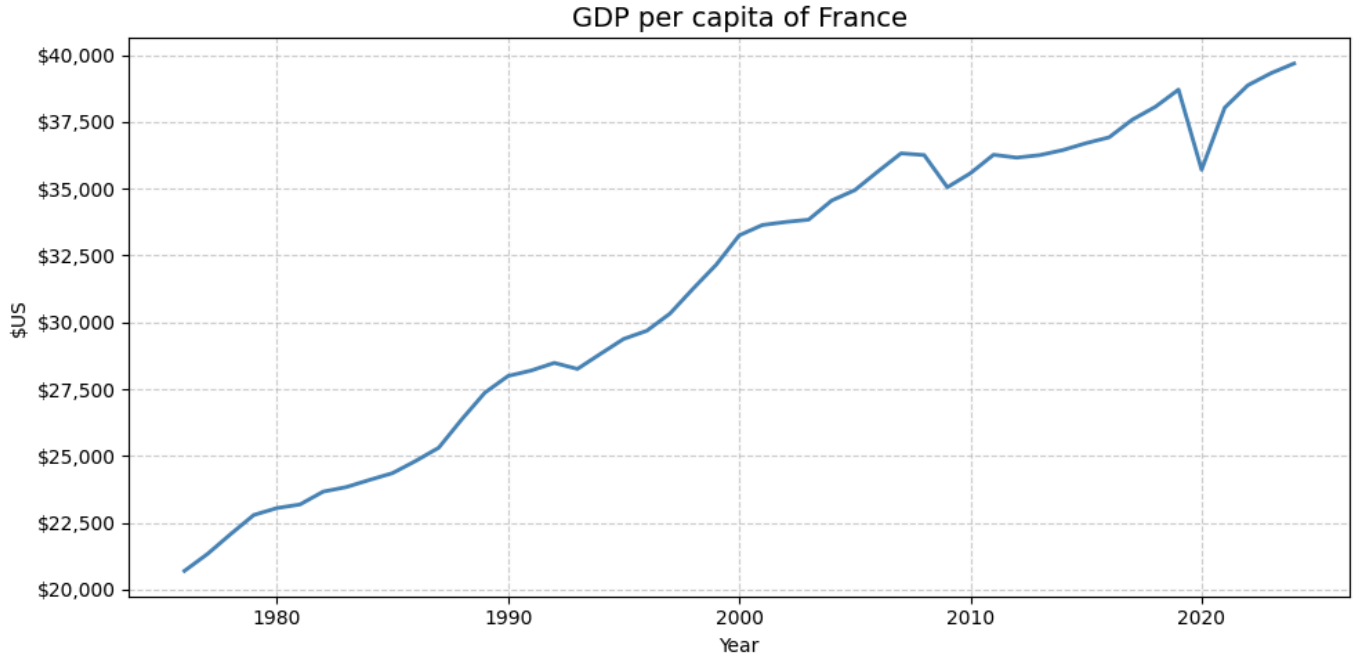
```
!pip install pmdarima statsmodels -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.seasonal import STL
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
from sklearn.metrics import mean_absolute_error, mean_squared_error
from scipy import stats
import pmdarima as pm
import warnings
warnings.filterwarnings('ignore')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 689.1/689.1 kB 7.9 MB/s eta 0:00:00
```

```
data = pd.read_excel("/content/france_gdp_per_capita_data.xlsx")
GDP_COL = 'France GDP per capita (constant US $)'
print(data.head())
plt.figure(figsize=(10, 5))
plt.plot(data['Year'], data[GDP_COL], color='steelblue', linewidth=2, markersize=4)
plt.title("GDP per capita of France", fontsize=14)
plt.xlabel("Year")
plt.ylabel("$US")
plt.gca().yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
print(data.describe())
```

```
   Year  France GDP per capita (constant US $)
0  1976                           20700.618623
1  1977                           21336.387782
2  1978                           22078.722816
3  1979                           22792.235950
4  1980                           23050.925595
```

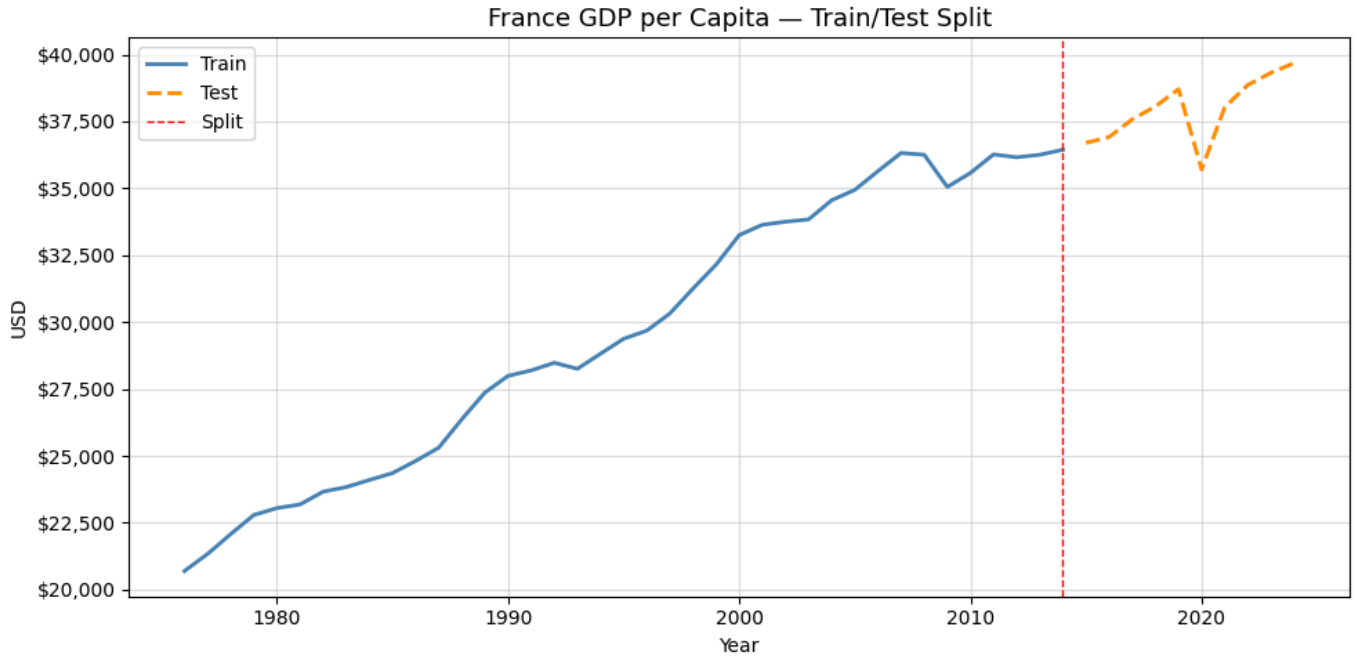



```
             Year  France GDP per capita (constant US $)
count    49.00000                              49.000000
mean   2000.00000                           31328.374694
std      14.28869                            5755.259771
min    1976.00000                           20700.618623
25%    1988.00000                           26374.122181
50%    2000.00000                           33255.137964
75%    2012.00000                           36257.365893
max    2024.00000                           39683.395349
```

```
SPLIT_YEAR = 2014
train = data[data['Year'] <= SPLIT_YEAR].copy()
test = data[data['Year'] > SPLIT_YEAR].copy()
h = len(test)
print(f"\nTrain: {train['Year'].min()} – {train['Year'].max()} ({len(train)} obs)")
print(f"Test:  {test['Year'].min()} – {test['Year'].max()} ({len(test)} obs)")
print(f"Forecast horizon h = {h}")

plt.figure(figsize=(10, 5))
plt.plot(train['Year'], train[GDP_COL], color='steelblue', linewidth=2, label='Train')
plt.plot(test['Year'],  test[GDP_COL],  color='darkorange', linewidth=2, linestyle='--', label='Test')
plt.axvline(SPLIT_YEAR, color='red', linestyle='--', linewidth=1, label='Split')
plt.title("France GDP per Capita — Train/Test Split", fontsize=13)
plt.xlabel("Year")
```

```
plt.ylabel("USD")
plt.gca().yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```

```
Train: 1976 – 2014 (39 obs)
Test:  2015 – 2024 (10 obs)
Forecast horizon h = 10
```




```
# Mean / Historical average method
# Forecast = average of training data
mean_value = float(train['France GDP per capita (constant US $)'].mean())
mean_fc = [round(mean_value, 3)] * len(test)
mean_fc
```

```
[29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409,
 29628.409]
```

```
# Naïve method
# Forecast = last observed value
naive_fc = [round(float(train['France GDP per capita (constant US $)'].iloc[-1]),3)]* len(test)
naive_fc
```

```
[36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395,
 36444.395]
```

```
# Drift Method
# Trend-based linear growth: Y_(T+h)=Y_T+((Y_T-Y_1)/(T-1))*h
y_T = train['France GDP per capita (constant US $)'].iloc[-1]   # last value
y_1 = train['France GDP per capita (constant US $)'].iloc[0]    # first value
T = len(train)
drift = (y_T - y_1) / (T - 1)
drift_fc = []

for step in range(1, len(test)+1):
    fc = round(float(y_T + step * drift),3)
    drift_fc.append(fc)
drift_fc
```

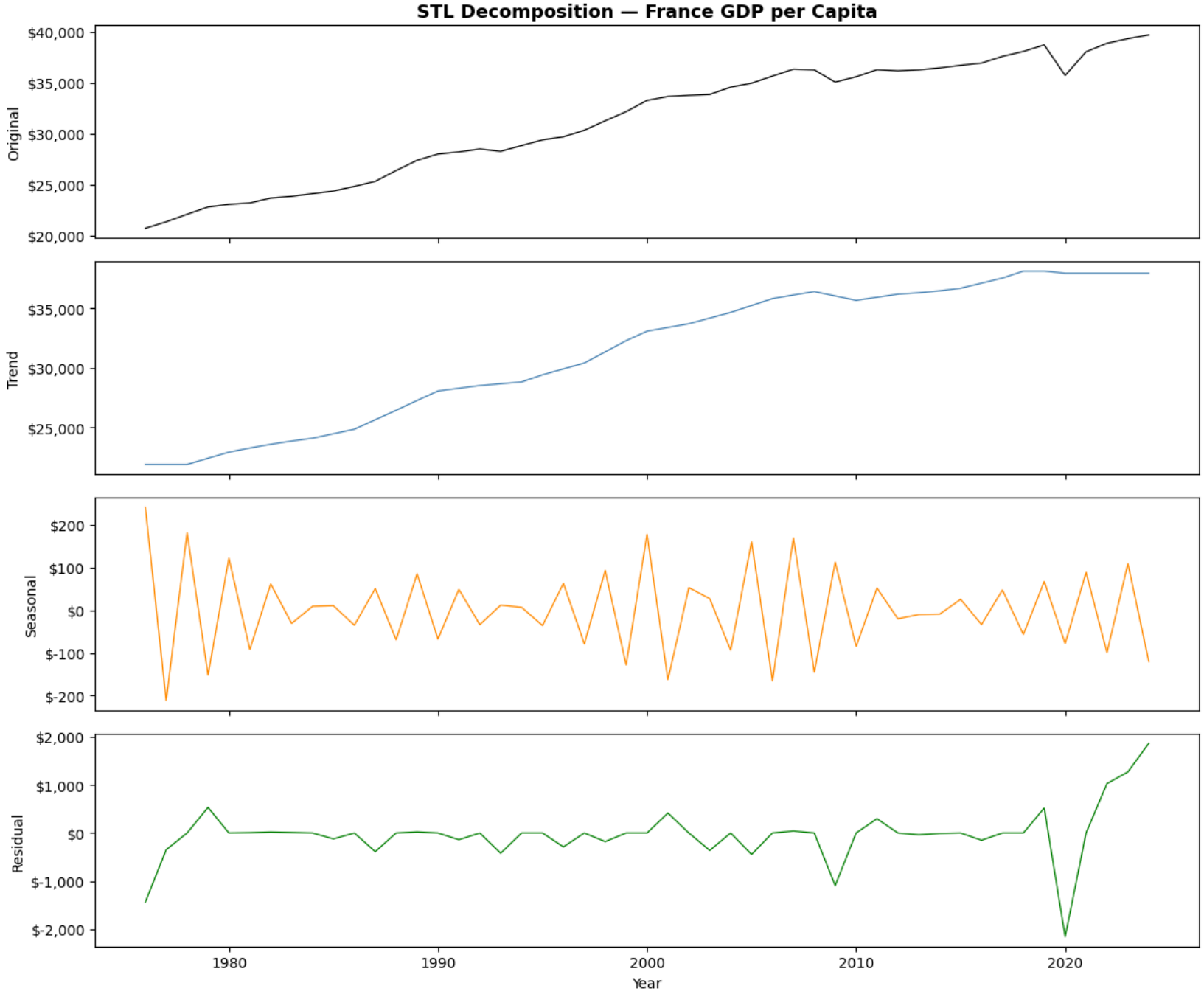
```
[36858.705,
 37273.015,
 37687.325,
 38101.635,
 38515.944,
 38930.254,
 39344.564,
 39758.874,
 40173.184,
 40587.494]
```

```
ets_model = ExponentialSmoothing(
    train[GDP_COL],
    trend='add'       # additive trend, no seasonality (annual data)
).fit()

ets_fc = round(ets_model.forecast(h),3).tolist()
print(ets_model.summary())
ets_fc
```

```
                            ExponentialSmoothing Model Results
==============================================================================================
Dep. Variable:     France GDP per capita (constant US $)   No. Observations:                   39
Model:                              ExponentialSmoothing   SSE                      6631500.894
Optimized:                                          True   AIC                          477.707
Trend:                                          Additive   BIC                          484.362
Seasonal:                                           None   AICC                         480.332
Seasonal Periods:                                   None   Date:                Mon, 13 Apr 2026
Box-Cox:                                           False   Time:                        18:37:06
```

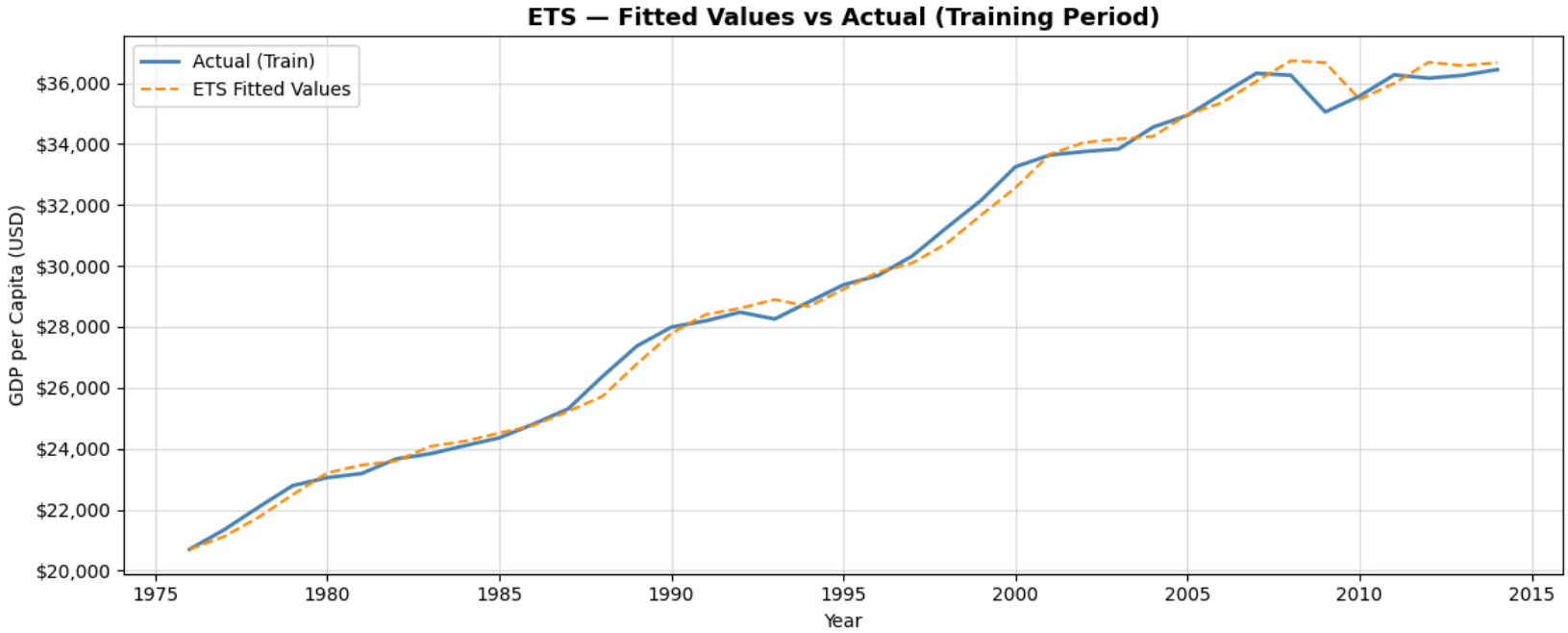
```
Box-Cox Coeff.:                              None
==============================================================================
                     coeff                 code              optimized
------------------------------------------------------------------------------
smoothing_level            1.0000000               alpha                 True
smoothing_trend             0.000000                beta                 True
initial_level              20286.281                 l.0                 True
initial_trend              414.28643                 b.0                 True
------------------------------------------------------------------------------
[36858.681,
 37272.968,
 37687.254,
 38101.541,
 38515.827,
 38930.114,
 39344.4,
 39758.686,
 40172.973,
 40587.259]
```

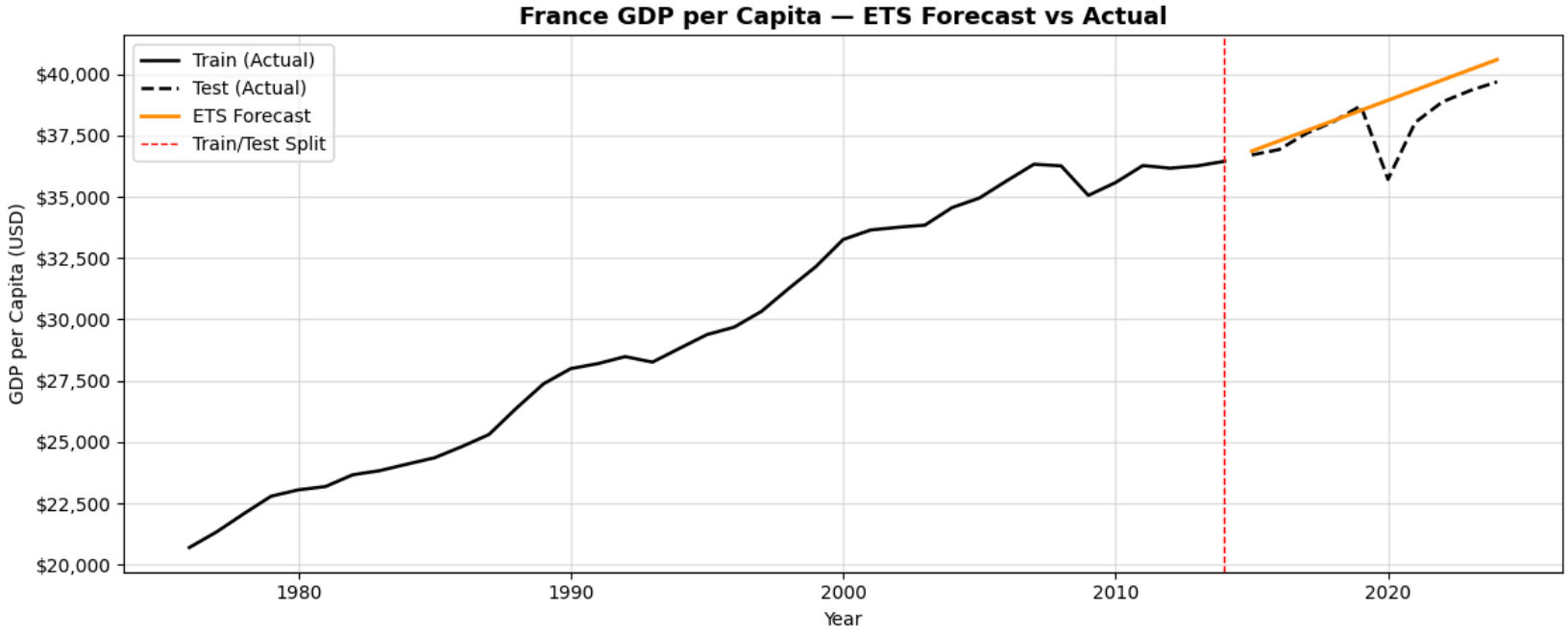
```
# STL DECOMPOSITION + DRIFT
# Original series = Trend + Seasonal + Residual

# For annual data period=2 is the minimum allowed
# Seasonal component will be very small — this is expected
stl      = STL(train[GDP_COL], period=2, robust=True).fit()
deseas   = train[GDP_COL].values - stl.seasonal
stl_drift = (deseas.iloc[-1] - deseas.iloc[0]) / (T - 1)
stl_base  = [deseas.iloc[-1] + i * stl_drift for i in range(1, h + 1)]
season_proj = np.tile(stl.seasonal.iloc[-2:], h // 2 + 1)[:h]
stl_fc    = (np.array(stl_base) + season_proj).tolist()

# Plot STL decomposition
stl_full = STL(data[GDP_COL], period=2, robust=True).fit()
fig, axes = plt.subplots(4, 1, figsize=(12, 10), sharex=True)
for ax, comp, lbl, col in zip(
    axes,
    [data[GDP_COL], stl_full.trend, stl_full.seasonal, stl_full.resid],
    ['Original', 'Trend', 'Seasonal', 'Residual'],
    ['black', 'steelblue', 'darkorange', 'green']
):
    ax.plot(data['Year'], comp, color=col, linewidth=0.9)
    ax.set_ylabel(lbl)
    ax.yaxis.set_major_formatter(
        mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
axes[0].set_title("STL Decomposition — France GDP per Capita",
                  fontsize=13, fontweight='bold')
axes[-1].set_xlabel("Year")
plt.tight_layout()
plt.show()
```




```
# PLOT FITTED VALUES vs ACTUAL (TRAINING PERIOD)
```

```
fitted_values = ets_model.fittedvalues

plt.figure(figsize=(12, 5))
plt.plot(train['Year'], train[GDP_COL],
         color='steelblue', linewidth=2, label='Actual (Train)')
plt.plot(train['Year'], fitted_values,
         color='darkorange', linewidth=1.5,
         linestyle='--', label='ETS Fitted Values')
plt.gca().yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
plt.title("ETS — Fitted Values vs Actual (Training Period)",
          fontsize=13, fontweight='bold')
plt.xlabel("Year")
plt.ylabel("GDP per Capita (USD)")
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()

# PLOT FORECAST vs ACTUAL (TEST PERIOD)

plt.figure(figsize=(12, 5))

plt.plot(train['Year'], train[GDP_COL],
         color='black', linewidth=1.8, label='Train (Actual)')
plt.plot(test['Year'], test[GDP_COL],
         color='black', linewidth=1.8,
         linestyle='--', label='Test (Actual)')

# ETS forecast
plt.plot(test['Year'], ets_fc,
         color='darkorange', linewidth=2,
         label='ETS Forecast')

plt.axvline(SPLIT_YEAR, color='red',
            linestyle='--', linewidth=1,
            label='Train/Test Split')

plt.gca().yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
plt.title("France GDP per Capita — ETS Forecast vs Actual",
          fontsize=13, fontweight='bold')
plt.xlabel("Year")
plt.ylabel("GDP per Capita (USD)")
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```


ETS — Fitted Values vs Actual (Training Period)
Actual (Train)
ETS Fitted Values
GDP per Capita (USD)
$36,000
$34,000
$32,000
$30,000
$28,000
$26,000
$24,000
$22,000
$20,000
1975
1980
1985
1990
1995
2000
2005
2010
2015
Year


France GDP per Capita — ETS Forecast vs Actual
Train (Actual)
Test (Actual)
ETS Forecast
Train/Test Split
GDP per Capita (USD)
$40,000
$37,500
$35,000
$32,500
$30,000
$27,500
$25,000
$22,500
$20,000
1980
1990
2000
2010
2020
Year

```
# ARIMA Model
```

```
# Box-Cox to justify log
gdp_boxcox, lambda_val = stats.boxcox(train[GDP_COL])
print(f"Box-Cox lambda: {lambda_val:.4f}")
print("→ lambda ≈ 1 means no transformation is needed")

# Use raw series with datetime index for ARIMA
series_full  = pd.Series(
    data[GDP_COL].values,
    index=pd.to_datetime(data['Year'], format='%Y')
)
train_series = series_full[series_full.index.year <= SPLIT_YEAR]
test_series  = series_full[series_full.index.year > SPLIT_YEAR]
series_full.head()
```

```
Box-Cox lambda: 1.1628
→ lambda ≈ 1 means no transformation is needed
```

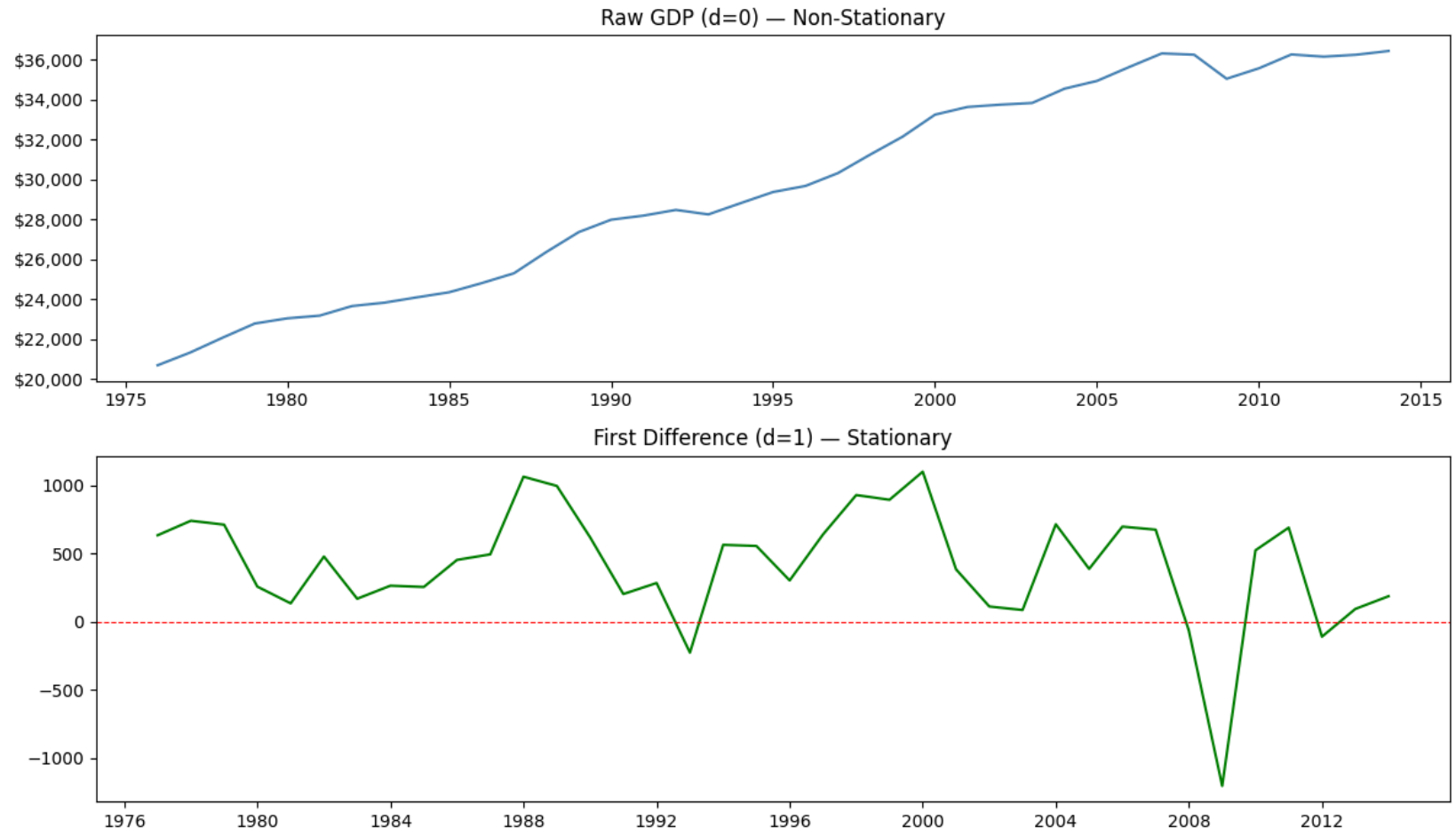
Year	0
1976-01-01	20700.618623
1977-01-01	21336.387782
1978-01-01	22078.722816
1979-01-01	22792.235950
1980-01-01	23050.925595

dtype: float64

```
def check_stationarity(ts, label="Series"):
    print(f"\n--- {label} ---")

    adf    = adfuller(ts.dropna())
    kpss_r = kpss(ts.dropna(), regression='c')
    print(f"ADF  p-value: {adf[1]:.4f}   → "
          f"{'Stationary' if adf[1] < 0.05 else 'Non-Stationary'}")
    print(f"KPSS p-value: {kpss_r[1]:.4f} → "
          f"{'Stationary' if kpss_r[1] > 0.05 else 'Non-Stationary'}")

# Test raw series
check_stationarity(train_series, "Raw GDP (d=0)")

# First difference
train_diff1 = train_series.diff().dropna()
check_stationarity(train_diff1, "First Difference (d=1)")
```

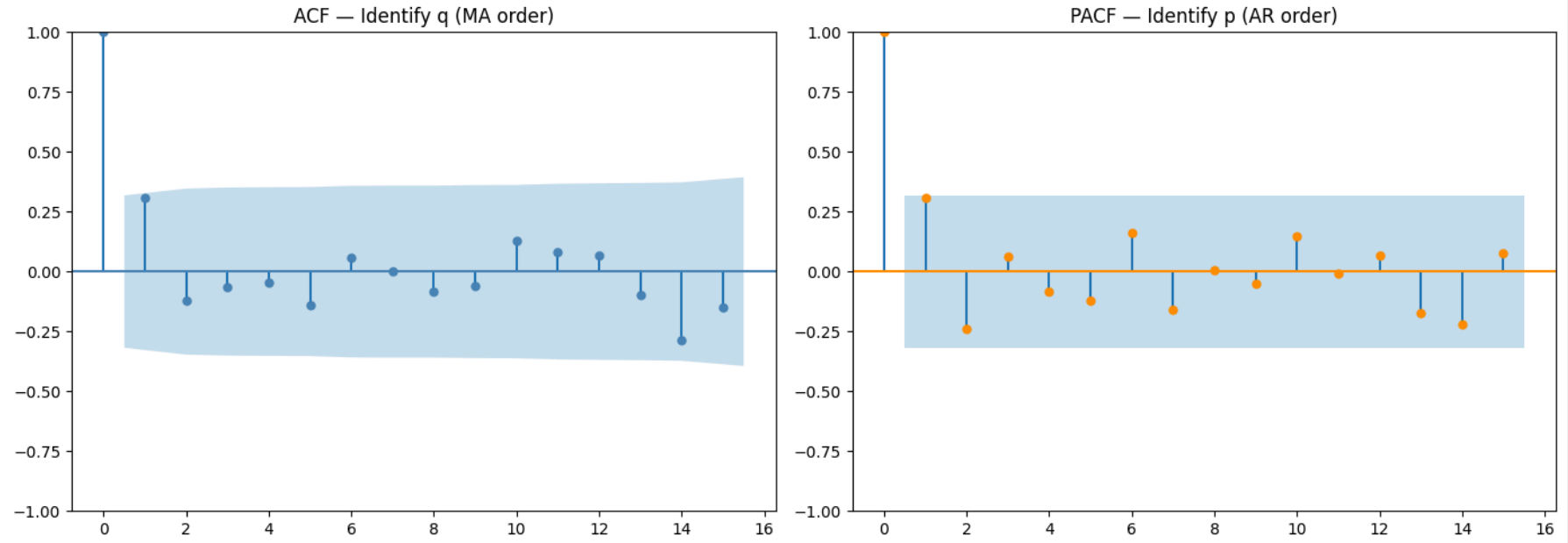
```
--- Raw GDP (d=0) ---
ADF  p-value: 0.4981   → Non-Stationary
KPSS p-value: 0.0100 → Non-Stationary

--- First Difference (d=1) ---
ADF  p-value: 0.0005   → Stationary
KPSS p-value: 0.1000 → Stationary
```

```
# Plot
fig, axes = plt.subplots(2, 1, figsize=(12, 7))
axes[0].plot(train_series, color='steelblue')
axes[0].set_title("Raw GDP (d=0) — Non-Stationary")
axes[0].yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
axes[1].plot(train_diff1, color='green')
axes[1].axhline(0, color='red', linestyle='--', linewidth=0.8)
axes[1].set_title("First Difference (d=1) — Stationary")
plt.tight_layout()
plt.show()
```




```
# ACF / PACF

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
plot_acf(train_diff1,  lags=15, ax=axes[0], color='steelblue')
axes[0].set_title("ACF — Identify q (MA order)")
plot_pacf(train_diff1, lags=15, ax=axes[1], color='darkorange', method='ywm')
```

```
axes[1].set_title("PACF — Identify p (AR order)")
plt.tight_layout()
plt.show()
```




```
d = 1
candidates = [(0,d,0), (0,d,1), (1,d,0), (1,d,1)]
ic_results = []

for order in candidates:
    try:
        m = ARIMA(train_series, order=order, trend='t').fit()
        ic_results.append({
            'Order': f"ARIMA{order}+drift",
            'AICc':  round(m.aicc, 3),
            'AIC':   round(m.aic,  3),
            'BIC':   round(m.bic,  3)
        })
    except:
        pass

ic_df = pd.DataFrame(ic_results).sort_values('AICc')
print("\nManual ARIMA Candidate Comparison:")
print(ic_df.to_string(index=False))
```

```
Manual ARIMA Candidate Comparison:
              Order    AICc     AIC     BIC
ARIMA(0, 1, 1)+drift 570.091 569.385 574.298
ARIMA(1, 1, 0)+drift 570.293 569.587 574.500
```

```
ARIMA(0, 1, 0)+drift 570.833 570.490 573.765
ARIMA(1, 1, 1)+drift 572.522 571.310 577.860
```

```
auto_model = pm.auto_arima(
    train_series,
    d=None,
    max_p=3, max_q=3,
    information_criterion='aicc',
    stepwise=True,
    seasonal=False,
    trace=True
)
print(auto_model.summary())
best_p, best_d, best_q = auto_model.order
print(f"\n Best order: ARIMA({best_p},{best_d},{best_q})")
```

```
Performing stepwise search to minimize aicc
 ARIMA(2,1,2)(0,0,0)[0] intercept   : AICC=577.895, Time=0.17 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AICC=570.823, Time=0.02 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AICC=570.253, Time=0.09 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AICC=570.053, Time=0.09 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AICC=594.625, Time=0.02 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AICC=572.644, Time=0.09 sec
 ARIMA(0,1,2)(0,0,0)[0] intercept   : AICC=572.571, Time=0.38 sec
 ARIMA(1,1,2)(0,0,0)[0] intercept   : AICC=575.151, Time=0.31 sec
 ARIMA(0,1,1)(0,0,0)[0]             : AICC=586.535, Time=0.05 sec

Best model:  ARIMA(0,1,1)(0,0,0)[0] intercept
Total fit time: 1.238 seconds
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   39
Model:               SARIMAX(0, 1, 1)   Log Likelihood                -281.684
Date:                Mon, 13 Apr 2026   AIC                            569.367
Time:                        18:37:49   BIC                            574.280
Sample:                  01-01-1976   HQIC                           571.115
                       - 01-01-2014
Covariance Type:                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept    397.0477     85.847      4.625      0.000     228.790     565.305
ma.L1          0.2097      0.133      1.577      0.115      -0.051       0.470
sigma2      1.508e+05   2.88e+04      5.228      0.000    9.43e+04    2.07e+05
===================================================================================
Ljung-Box (L1) (Q):                   0.66   Jarque-Bera (JB):                23.80
Prob(Q):                              0.42   Prob(JB):                         0.00
Heteroskedasticity (H):               3.04   Skew:                            -1.19
Prob(H) (two-sided):                  0.05   Kurtosis:                         6.06
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

 Best order: ARIMA(0,1,1)
```

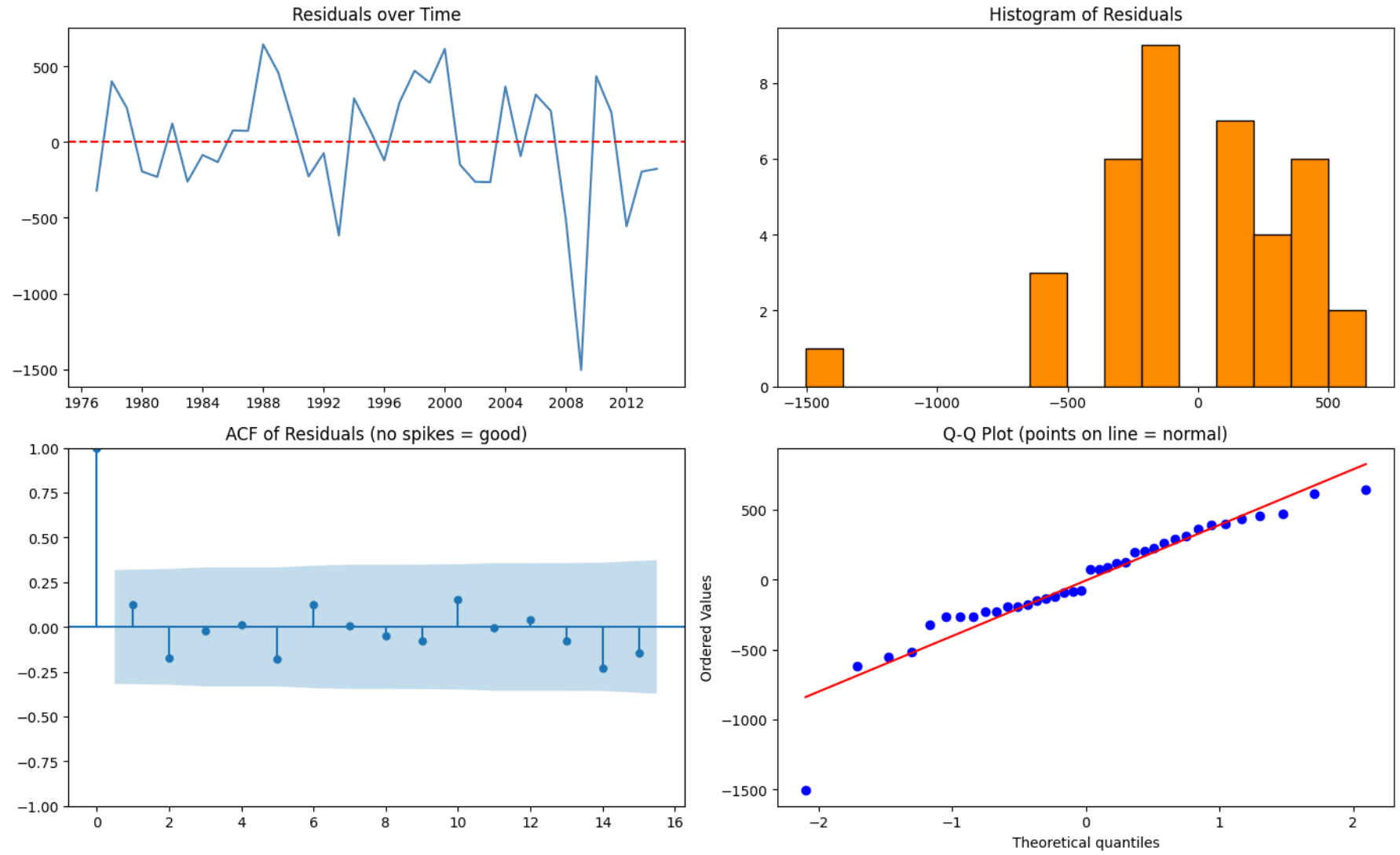
```
arima_model = ARIMA(
    train_series,
    order=(best_p, best_d, best_q),
```

```
    trend='t'
).fit()

print(arima_model.summary())
# Forecast on test period — no back-transformation needed
arima_fc = round(arima_model.forecast(steps=h),3).tolist()
arima_fc
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   39
Model:                 ARIMA(0, 1, 1)   Log Likelihood                -281.693
Date:                Mon, 13 Apr 2026   AIC                            569.385
Time:                        18:37:54   BIC                            574.298
Sample:                    01-01-1976   HQIC                           571.133
                         - 01-01-2014
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
x1           406.9396     85.936      4.735      0.000     238.508     575.372
ma.L1          0.2092      0.133      1.573      0.116      -0.051       0.470
sigma2      1.496e+05   2.85e+04      5.242      0.000    9.37e+04    2.06e+05
===================================================================================
Ljung-Box (L1) (Q):                   0.67   Jarque-Bera (JB):                23.89
Prob(Q):                              0.41   Prob(JB):                         0.00
Heteroskedasticity (H):               3.11   Skew:                            -1.19
Prob(H) (two-sided):                  0.05   Kurtosis:                         6.06
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[36814.158,
 37221.098,
 37628.037,
 38034.977,
 38441.917,
 38848.856,
 39255.796,
 39662.736,
 40069.675,
 40476.615]
```

```
# ARIMA RESIDUAL DIAGNOSTICS

residuals = arima_model.resid[1:]

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

axes[0,0].plot(residuals, color='steelblue')
axes[0,0].axhline(0, color='red', linestyle='--')
axes[0,0].set_title("Residuals over Time")

axes[0,1].hist(residuals, bins=15, color='darkorange', edgecolor='black')
axes[0,1].set_title("Histogram of Residuals")

plot_acf(residuals, lags=15, ax=axes[1,0])
```

```
axes[1,0].set_title("ACF of Residuals (no spikes = good)")

stats.probplot(residuals, plot=axes[1,1])
axes[1,1].set_title("Q-Q Plot (points on line = normal)")

plt.suptitle("ARIMA Residual Diagnostics", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

lb = acorr_ljungbox(residuals, lags=[10], return_df=True)
print("\nLjung-Box Test (H0: no autocorrelation in residuals):")
print(lb)
print("p > 0.05 → Residuals are white noise — model is well specified"
      if lb['lb_pvalue'].values[0] > 0.05
      else "p < 0.05 → Residuals still have structure")
```




```
Ljung-Box Test (H0: no autocorrelation in residuals):
     lb_stat  lb_pvalue
10  5.903016   0.823345
p > 0.05 → Residuals are white noise — model is well specified
```

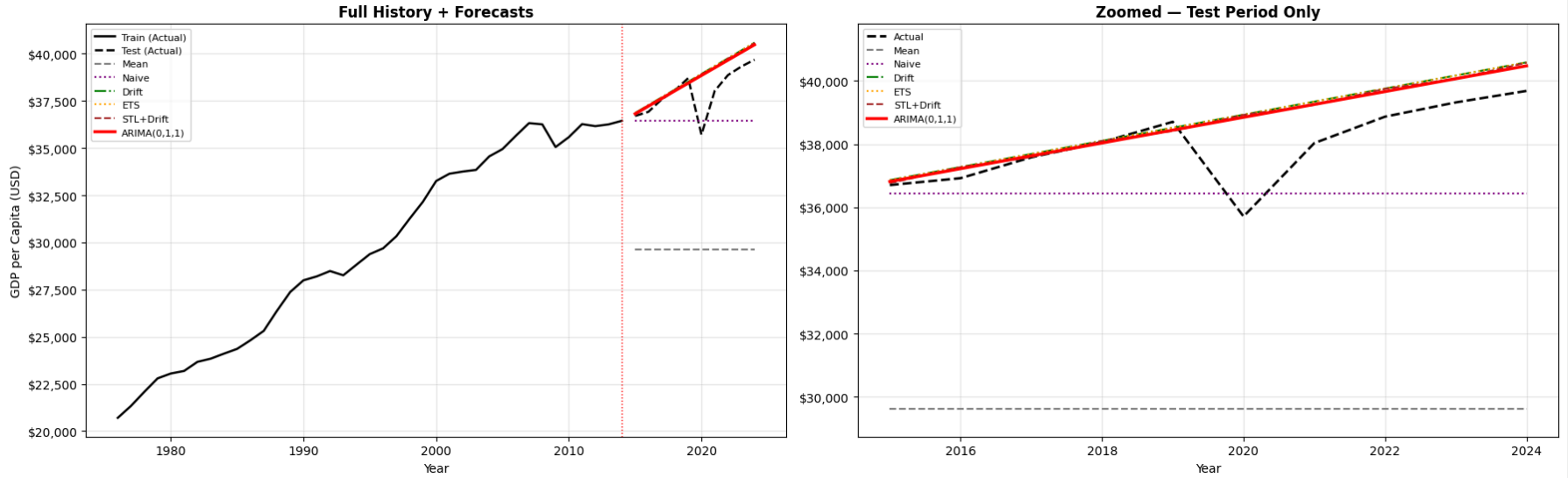
```
# ACCURACY COMPARISON — ALL MODELS

actual = test[GDP_COL].values
```

```
def get_metrics(actual, predicted, name):
    mae  = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    mape = np.mean(np.abs(
        (np.array(actual) - np.array(predicted)) / np.array(actual)
    )) * 100
    return {
        'Model':    name,
        'MAE':      round(mae,  2),
        'RMSE':     round(rmse, 2),
        'MAPE (%)': round(mape, 2)
    }

rows = [
    get_metrics(actual, mean_fc,  'Mean'),
    get_metrics(actual, naive_fc, 'Naive'),
    get_metrics(actual, drift_fc, 'Drift'),
    get_metrics(actual, ets_fc,   'ETS'),
    get_metrics(actual, stl_fc,   'STL+Drift'),
    get_metrics(actual, arima_fc, f'ARIMA({best_p},{best_d},{best_q})'),
]

acc_df = pd.DataFrame(rows).sort_values('RMSE')
print("\n=== Model Accuracy Comparison ===")
print(acc_df.to_string(index=False))

best_model_name = acc_df.iloc[0]['Model']
print(f"\n Best Model: {best_model_name}")
```

```
=== Model Accuracy Comparison ===
       Model     MAE    RMSE  MAPE (%)
ARIMA(0,1,1)  745.30 1155.69      2.00
   STL+Drift  768.41 1188.76      2.06
         ETS  802.60 1209.76      2.15
       Drift  802.71 1209.87      2.15
       Naive 1660.88 1923.62      4.30
        Mean 8329.83 8413.96     21.87

 Best Model: ARIMA(0,1,1)
```

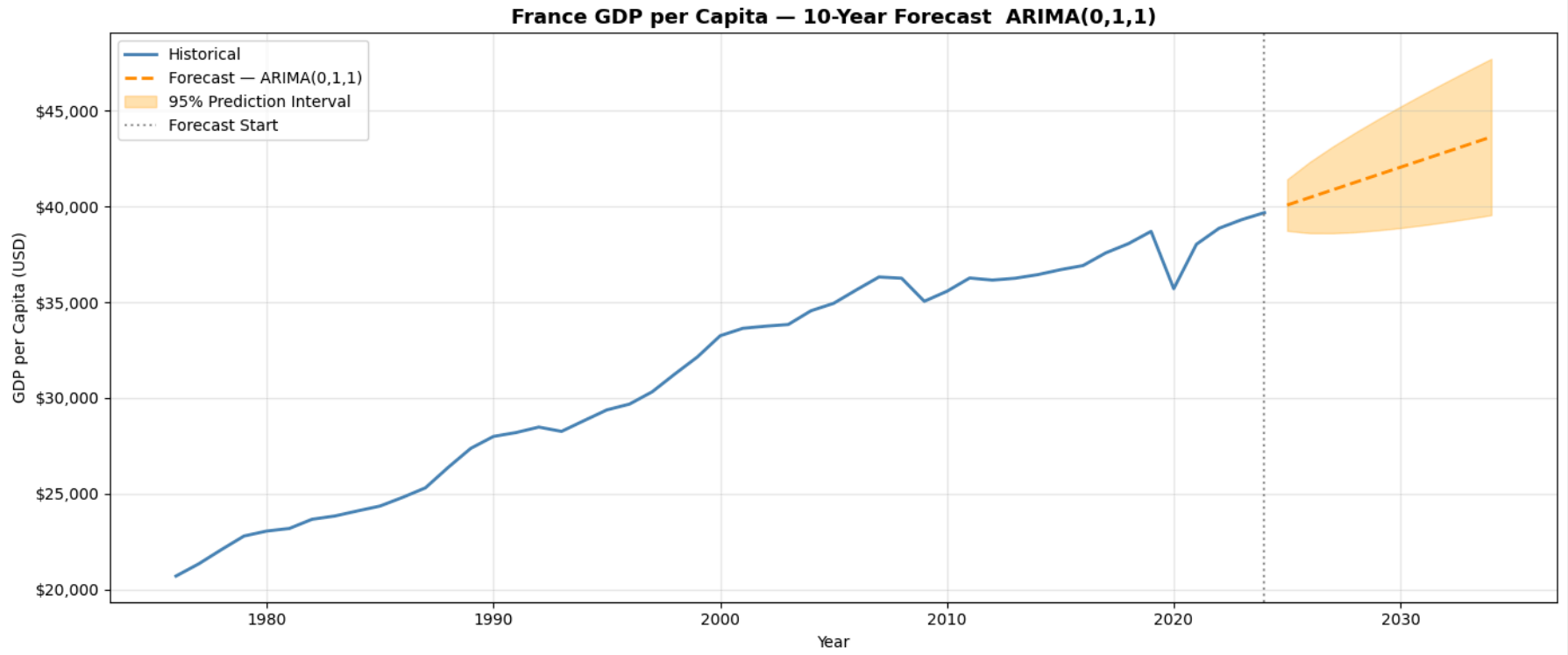
```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# --- Left plot: Full history ---
axes[0].plot(train['Year'], train[GDP_COL],
             color='black', linewidth=1.8, label='Train (Actual)')
axes[0].plot(test['Year'], test[GDP_COL],
             color='black', linewidth=1.8, linestyle='--', label='Test (Actual)')
axes[0].plot(test['Year'], mean_fc,
             color='gray',   lw=1.5, linestyle='--', label='Mean')
axes[0].plot(test['Year'], naive_fc,
             color='purple', lw=1.5, linestyle=':', label='Naive')
axes[0].plot(test['Year'], drift_fc,
             color='green',  lw=1.5, linestyle='-.', label='Drift')
axes[0].plot(test['Year'], ets_fc,
             color='orange', lw=1.5, linestyle=':', label='ETS')
```

```
axes[0].plot(test['Year'], stl_fc,
             color='brown',  lw=1.5, linestyle='--', label='STL+Drift')
axes[0].plot(test['Year'], arima_fc,
             color='red', lw=2.5, label=f'ARIMA({best_p},{best_d},{best_q})')
axes[0].axvline(SPLIT_YEAR, color='red', linestyle=':', linewidth=1)
axes[0].yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
axes[0].set_title("Full History + Forecasts", fontweight='bold')
axes[0].set_xlabel("Year")
axes[0].set_ylabel("GDP per Capita (USD)")
axes[0].legend(fontsize=8, loc='upper left')
axes[0].grid(True, alpha=0.3)

# --- Right plot: Zoomed into test period only ---
test_years = test['Year'].values
y_min = min(min(mean_fc), min(naive_fc), min(drift_fc),
            min(ets_fc), min(stl_fc), min(arima_fc),
            test[GDP_COL].min()) * 0.97
y_max = max(max(mean_fc), max(naive_fc), max(drift_fc),
            max(ets_fc), max(stl_fc), max(arima_fc),
            test[GDP_COL].max()) * 1.03

axes[1].plot(test['Year'], test[GDP_COL],
             color='black', linewidth=2, linestyle='--', label='Actual')
axes[1].plot(test['Year'], mean_fc,
             color='gray',   lw=1.5, linestyle='--', label='Mean')
axes[1].plot(test['Year'], naive_fc,
             color='purple', lw=1.5, linestyle=':', label='Naive')
axes[1].plot(test['Year'], drift_fc,
             color='green',  lw=1.5, linestyle='-.', label='Drift')
axes[1].plot(test['Year'], ets_fc,
             color='orange', lw=1.5, linestyle=':', label='ETS')
axes[1].plot(test['Year'], stl_fc,
             color='brown',  lw=1.5, linestyle='--', label='STL+Drift')
axes[1].plot(test['Year'], arima_fc,
             color='red', lw=2.5, label=f'ARIMA({best_p},{best_d},{best_q})')
axes[1].set_ylim(y_min, y_max)
axes[1].yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
axes[1].set_title("Zoomed — Test Period Only", fontweight='bold')
axes[1].set_xlabel("Year")
axes[1].legend(fontsize=8, loc='upper left')
axes[1].grid(True, alpha=0.3)

plt.suptitle("France GDP per Capita — All Models vs Actual",
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```




```
final_model = ARIMA(
    series_full,
    order=(best_p, best_d, best_q),
    trend='t'
).fit()

fc_result   = final_model.get_forecast(steps=10)
fc_mean     = fc_result.predicted_mean
fc_ci       = fc_result.conf_int(alpha=0.05)
fc_lower    = fc_ci.iloc[:, 0]
fc_upper    = fc_ci.iloc[:, 1]

last_year    = int(data['Year'].iloc[-1])
future_years = list(range(last_year + 1, last_year + 11))

# Plot
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(data['Year'], data[GDP_COL],
        color='steelblue', linewidth=2, label='Historical')
ax.plot(future_years, fc_mean.values,
        color='darkorange', linewidth=2, linestyle='--',
        label=f'Forecast — ARIMA({best_p},{best_d},{best_q})')
ax.fill_between(future_years, fc_lower.values, fc_upper.values,
                alpha=0.3, color='orange', label='95% Prediction Interval')
ax.axvline(x=last_year, color='gray', linestyle=':', alpha=0.8,
           label='Forecast Start')
ax.yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'${x:,.0f}'))
ax.set_title(
    f"France GDP per Capita — 10-Year Forecast  ARIMA({best_p},{best_d},{best_q})",
    fontsize=13, fontweight='bold')
```

```
ax.set_xlabel("Year")
ax.set_ylabel("GDP per Capita (USD)")
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
fc_df = pd.DataFrame({
    'Year':      future_years,
    'Forecast':  fc_mean.values.round(2),
    'Lower 95%': fc_lower.values.round(2),
    'Upper 95%': fc_upper.values.round(2)
})
print("\n=== 10-Year Forecast ===")
print(fc_df.to_string(index=False))
```

```
=== 10-Year Forecast ===
 Year  Forecast  Lower 95%  Upper 95%
 2025  40080.10   38737.14   41423.06
 2026  40475.57   38617.48   42333.66
 2027  40871.05   38612.41   43129.69
 2028  41266.52   38668.37   43864.68
 2029  41662.00   38763.83   44560.17
 2030  42057.47   38887.56   45227.38
```

```
2031  42452.95   39032.81   45873.08
2032  42848.42   39195.17   46501.68
2033  43243.90   39371.53   47116.26
2034  43639.37   39559.64   47719.10
```